

Experiment 'p_4_2_a_min_max_seq' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_seq

Natural language statement: Write a method that takes a sequence of integers and returns the smallest and largest of the inputs.

Method signature: p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

Ensures

- forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
- forall i :: 0 <= i < |inputs| ==> max >= inputs[i]
- exists i :: 0 <= i < |inputs| && min == inputs[i]
- exists i :: 0 <= i < |inputs| && max == inputs[i]

Requires

- 0 < |inputs|

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the smallest and largest of
→ the inputs.

The signature should be:

p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

The method should respect the following contract:

requires 0 < |inputs|, ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i],
→ ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i], ensures exists i :: 0
→ <= i < |inputs| && min == inputs[i], ensures exists i :: 0 <= i < |inputs| && max
→ == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dme4pWxueOvzeC3iHYBG7QaT8afNw

Model: gpt-5.5-2026-04-23

Created at: 1780486171

Finish reason: stop

Usage: 422 tokens in, and 1613 tokens out

```
{
  min := inputs[0];
  max := inputs[0];

  ghost var minIndex := 0;
  ghost var maxIndex := 0;
  var i := 1;

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= minIndex < i
    invariant 0 <= maxIndex < i
    invariant min == inputs[minIndex]
    invariant max == inputs[maxIndex]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    decreases |inputs| - i
```

```

{
  if inputs[i] < min {
    min := inputs[i];
    minIndex := i;
  }

  if inputs[i] > max {
    max := inputs[i];
    maxIndex := i;
  }

  i := i + 1;
}
}

```

Final Program

```

method p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)
  requires 0 < |inputs|
  ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
  ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i]
  ensures exists i :: 0 <= i < |inputs| && min == inputs[i]
  ensures exists i :: 0 <= i < |inputs| && max == inputs[i]
{
  min := inputs[0];
  max := inputs[0];

  ghost var minIndex := 0;
  ghost var maxIndex := 0;
  var i := 1;

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= minIndex < i
    invariant 0 <= maxIndex < i
    invariant min == inputs[minIndex]
    invariant max == inputs[maxIndex]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] < min {
      min := inputs[i];
      minIndex := i;
    }

    if inputs[i] > max {
      max := inputs[i];
      maxIndex := i;
    }
  }
}

```

```
    i := i + 1;  
  }  
}
```

Total Token Usage

Input tokens: 422

Output tokens: 1613

Reasoning tokens: 1361

Sum of ‘total tokens’: 2035

Experiment Timings

Overall Experiment started at 1780486170881, ended at 1780486210996, lasting 40115ms (40.12 seconds)

Iteration #1 started at 1780486170888, ended at 1780486210996, lasting 40108ms (40.11 seconds)